COMBAT · AIM · 합본

커서로 몹을 찍었는데 왜 빗나갈까

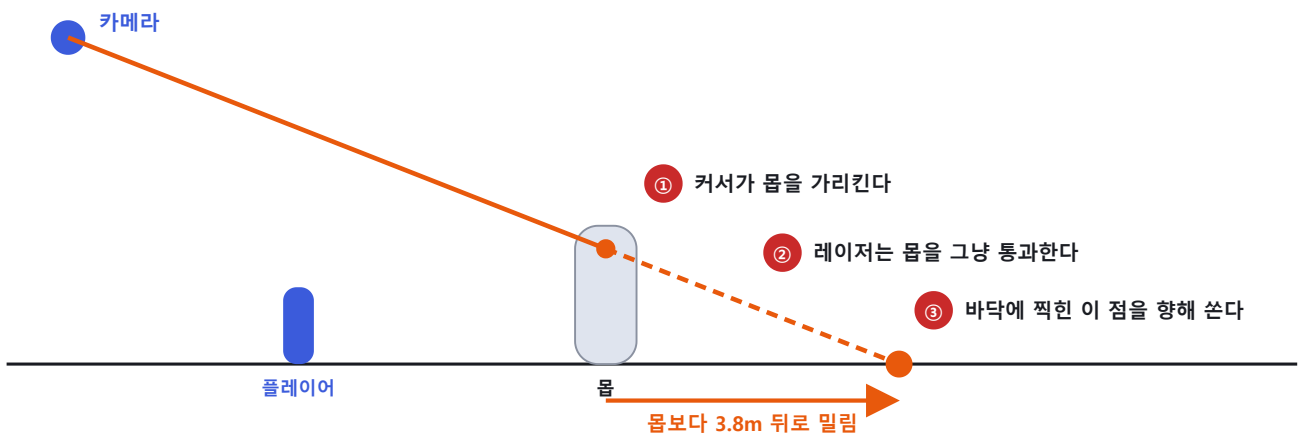
쿼터뷰에서 커서로 조준할 때 생기는 문제, 해결 방법, 실무 사례를 한 문서로 정리했다.

읽는 법 — 주제마다 `쉽게`를 먼저 읽고, 식과 정확한 그림이 궁금하면 `자세히`를 본다. 자세히는 건너뛰어도 흐름이 이어진다. 그림 속 숫자는 이해를 돕는 예시값이고, `실무에선`은 공개 자료(GDC 발표·개발사 블로그) 요약이다.

1. 지금 무슨 일이 일어나나

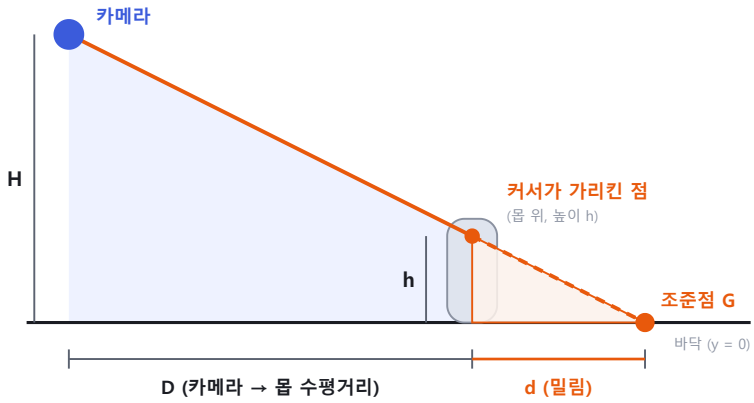
`쉽게`

카메라에서 커서 쪽으로 **레이저**를 쏜다고 생각하자. 게임은 이 레이저가 **바닥에 찍힌 점**을 조준점으로 쓰고, 캐릭터는 그 점을 향해 돌아서 쏜다. 문제는 레이저가 몹을 무시하고 **몹 뒤 바닥까지** 간다는 것이다.



`자세히`

밀리는 거리는 닮은 삼각형으로 구한다



닮은 삼각형 두 개

큰 삼각형 (파랑) : 높이 H, 밑변 D + d

작은 삼각형 (주황) : 높이 h, 밑변 d

$$h / d = H / (D + d)$$

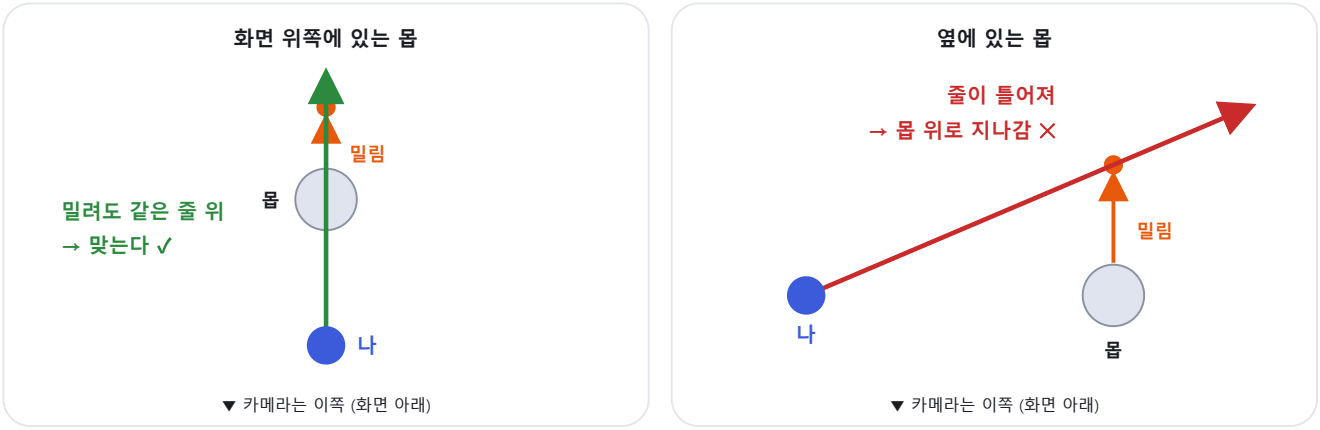
$$d = h \cdot D / (H - h)$$

- h ↑ (몹이 클수록, 머리를 찍을수록) → d ↑
- H ↓ (카메라가 낮을수록) → d ↑
- h → H 이면 분모 → 0, d → ∞

레이가 수평이 되어 바닥에 닿지 않는다

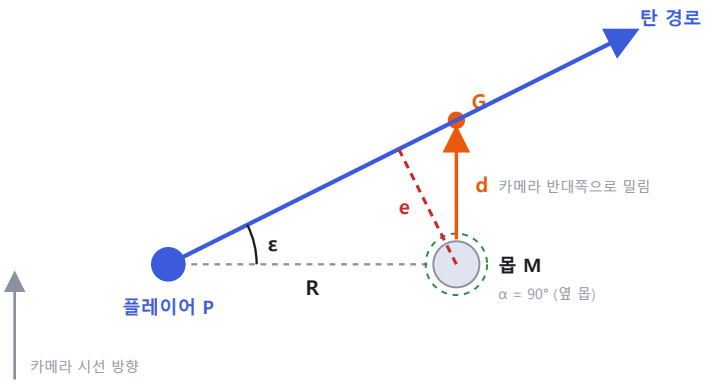
쉽게

밀리는 방향은 언제나 **카메라 반대쪽(화면 위쪽)**이다. 화면 위쪽 몹은 밀려도 같은 줄 위라 맞고, **옆에 있는 몹**은 쏘는 방향이 틀어져서 빗나간다.



자세히

밀린 거리 d가 방향 오차 ε로 바뀐다



옆에 있는 몹일수록 방향이 틀어진다

$$\varepsilon = \operatorname{atan}\left(d \cdot \sin \alpha \,/\, (R + d \cdot \cos \alpha) \right)$$

$$e = R \cdot \sin \varepsilon$$

명중 ⇔ $e < r_{몹} + r_{탄}$

α = P→M 과 밀림 방향(카메라 시선) 사이 각

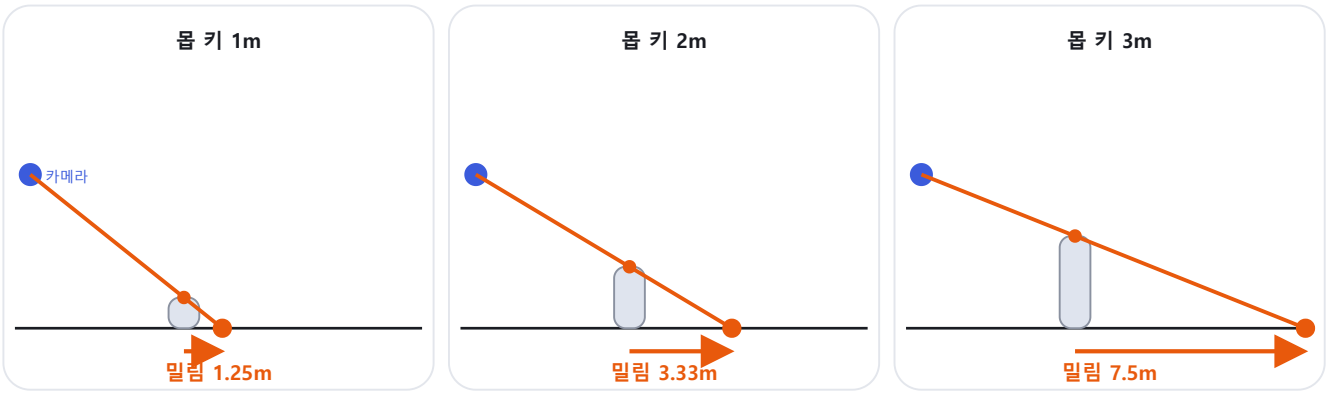
- α = 0° (화면 위쪽 몹): ε = 0
  거리만 밀리고 방향은 그대로 → 맞음
- α = 90° (옆 몹): ε = atan(d / R) 로 최대
- α = 180° (카메라 쪽 몹) 이고 d > R 이면
  G가 플레이어 뒤로 넘어가 정반대로 쏜다

2. 몹이 크면 왜 더 심해질까

쉽게

큰 몹의 머리처럼 **높은 곳**을 찍을수록 레이저가 옆으로 눕는다. 누운 레이저는 바닥에 닿기까지 훨씬 멀리 간다. 그래서 키가 조금만 커져도 밀림은 **확** 커진다.

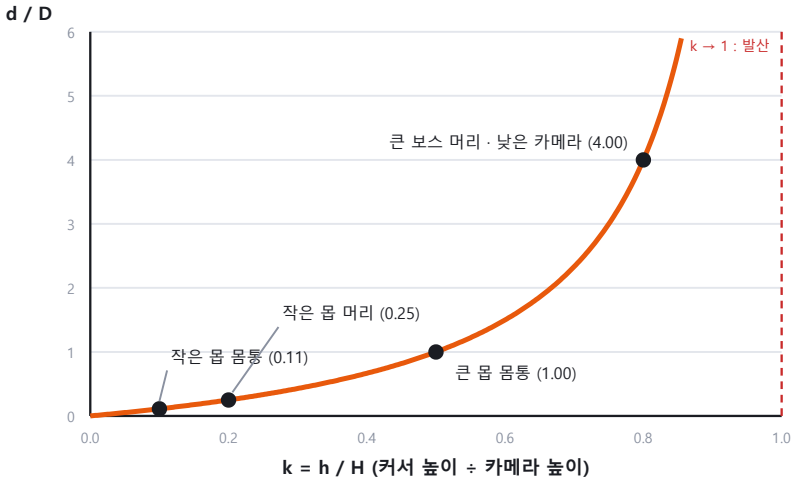
카메라 높이 5m, 몹까지 5m. 몹의 머리 끝을 찍었을 때



몹 키	조준점이 밀리는 거리	1m 몹과 비교
1m	1.25m	1배
2m	3.33m	2.67배
3m	7.5m	6배
4m	20m	16배
5m	레이저가 바닥에 닿지 않음 → 조준 실패	

자세히

밀림 비율은 k = (커서 높이 ÷ 카메라 높이) 하나로 정해진다



카메라·몹 크기와 상관없는 한 장

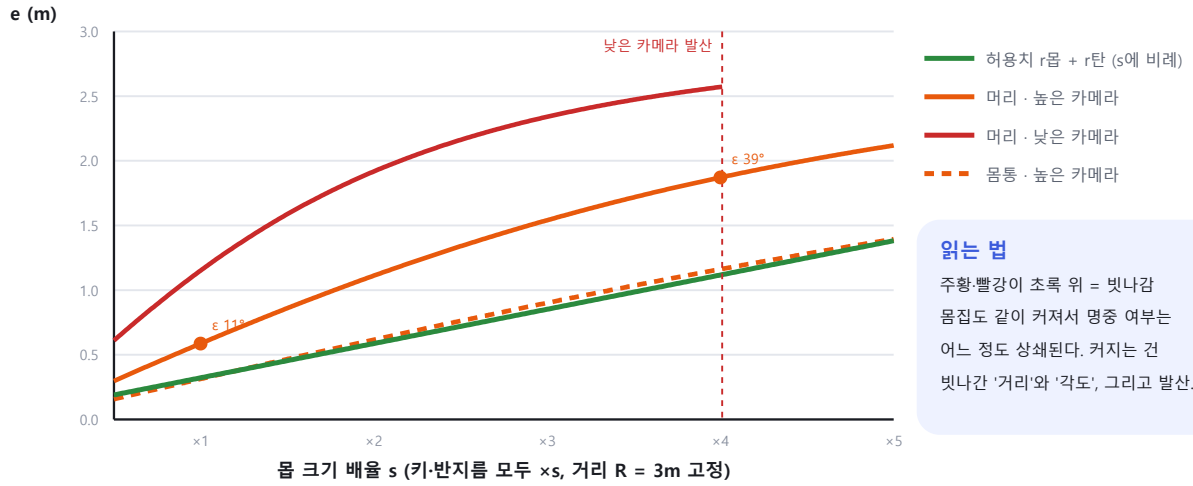
$$d / D = k / (1 - k)$$

앞 식을 H로 나누면 변수가 k 하나만 남는다.

- k = 0.2 → 몹까지 거리의 25% 밀림
- k = 0.5 → 100% (몹 거리만큼 더)
- k = 0.8 → 400%

선형이 아니라 쌍곡선이다.
작을 때는 거의 평평하다가
k가 1에 다가가면 수직으로 꺾인다.

몹 크기를 키우면 (옆 몹, 예시값)



몹 크기 배율 s (키·반지름 모두 ×s, 거리 R = 3m 고정)

읽는 법

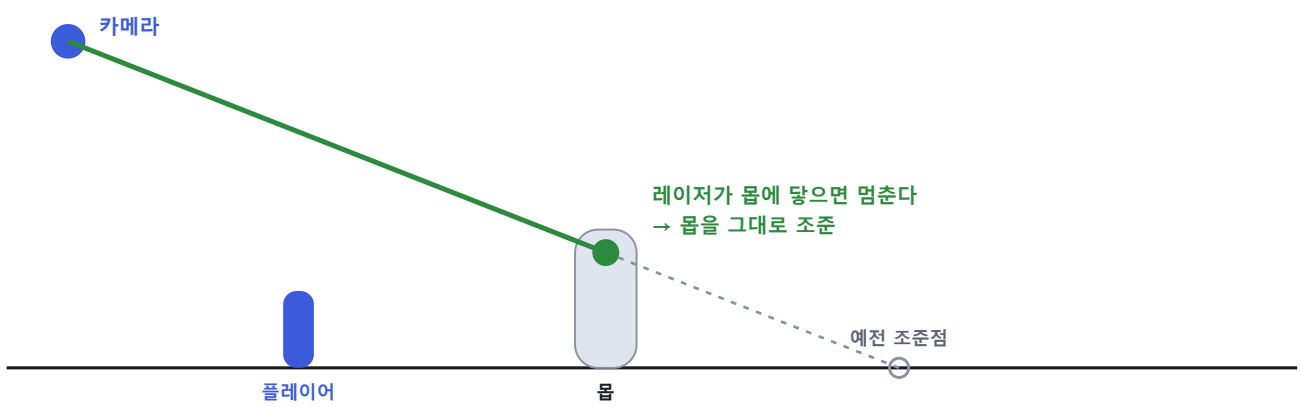
주황·빨강이 초록 위 = 빗나감
몸집도 같이 커져서 명중 여부는
어느 정도 상쇄된다. 커지는 건
빗나간 '거리'와 '각도', 그리고 발산.

정리

1. 키가 2배면 밀림은 **2.7배**, 3배면 **6배**. 직선이 아니라 쌍곡선으로 커진다.
2. 큰 몹은 몸도 커서 "완전히 빗나감"보다 **"엉뚱한 데로 쏨"**으로 느껴진다. 머리 조준 ×1에서 11°, ×4에서 39° 틀어진다.
3. 몹 키가 카메라 높이에 가까워지면 레이저가 바닥에 닿지 않아 **조준 자체가 실패**한다. 큰 보스 + 낮은 카메라가 이 경우다.

3. 가장 쉬운 해결: 레이저가 몹에도 닿게

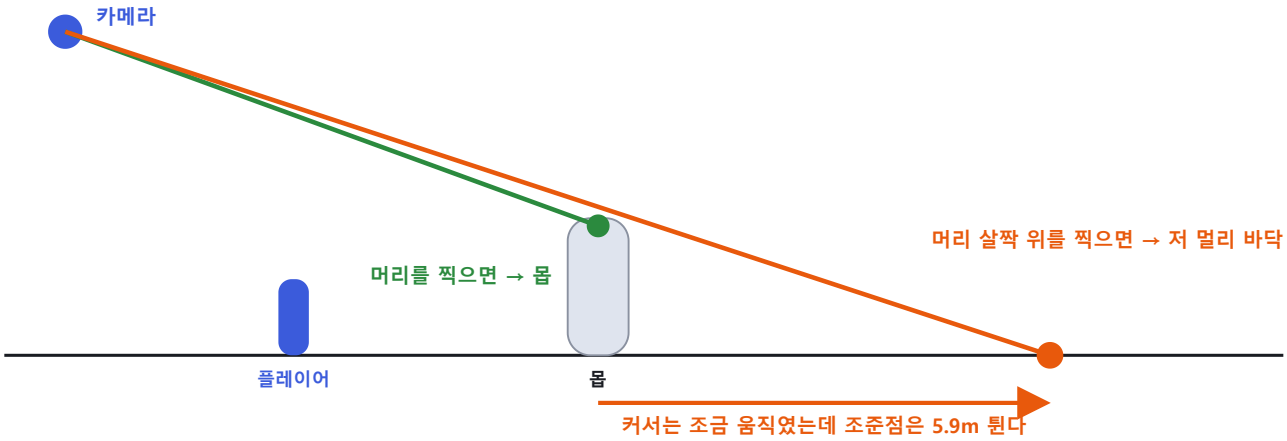
쉽게



커서가 몹 위에 있으면 바닥까지 가지 않고 **몹을 바로 조준**한다. 이것만으로 대부분 좋아지고 실제 게임도 쓰는 출발점이다. 하지만 네 가지 문제가 남는다.

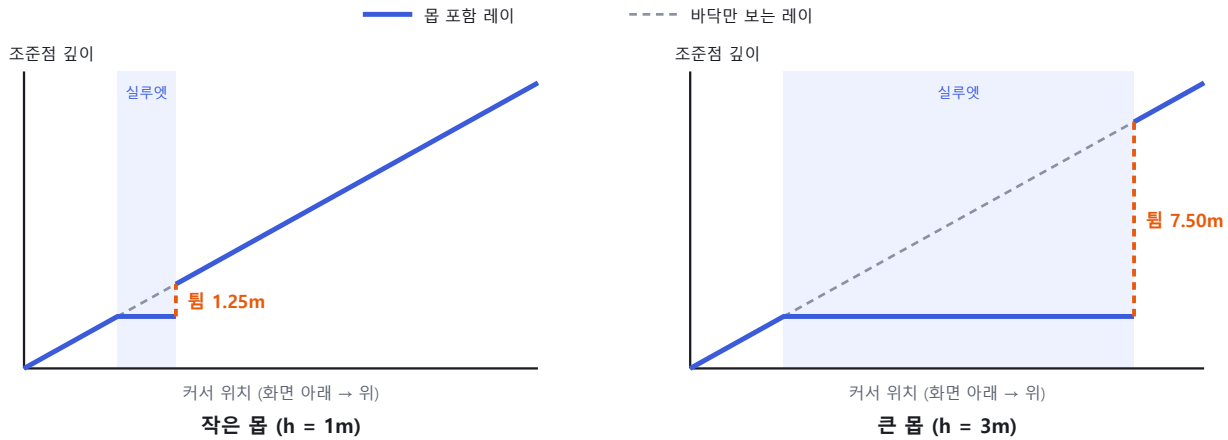
남는 문제 1 · 몹 머리 바로 위를 찍으면 확 튄다

쉽게



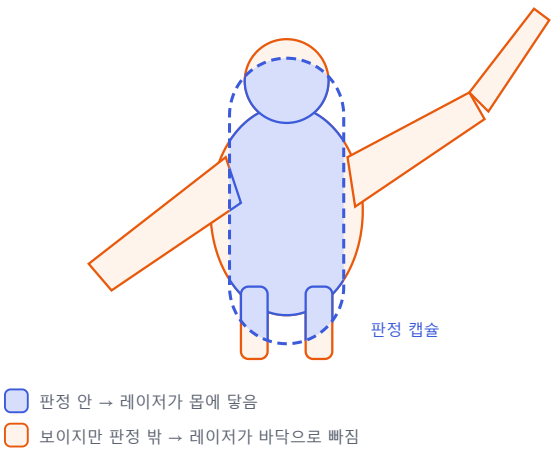
몹 위면 몹, 조금만 벗어나면 다시 저 뒤 바닥. **원래 문제가 없어진 게 아니라 몹 테두리로 모였을 뿐**이다.

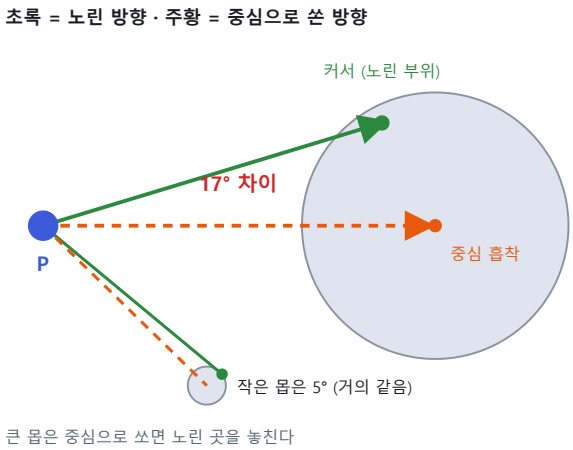
자세히



튀는 크기 = 몹 꼭대기 높이에서의 원래 밀림 = h·D/(H−h). 작은 몹 1.25m, 큰 몹 7.50m. 몹이 클수록 더 크게 튄다.

남는 문제 2 · 보이는 모양 ≠ 판정 모양 / 남는 문제 3 · 큰 몹은 "어디를" 찍었나





왼쪽: 게임은 몹을 단순한 캡슐로 판정한다. 팔·무기처럼 캡슐 밖으로 나온 곳을 찍으면 레이저가 다시 바닥으로 빠진다.

오른쪽: 작은 몹은 중심을 쏴도 5° 차이라 상관없다. 큰 보스의 어깨를 찍었는데 중심으로 쏘면 17° 벗어난다. 차이 = atan(커서가 중심에서 떨어진 거리 ÷ 몹까지 거리).

남는 문제 4 · 몹 위를 찍었다고 그 몹을 노린 건 아니다





뒤의 몹을 노렸거나 몹 뒤 바닥에 장판을 깔고 싶었을 수도 있다. 커서 아래 몹을 무조건 고르면 **"조준을 뺏겼다"**는 느낌이 든다.

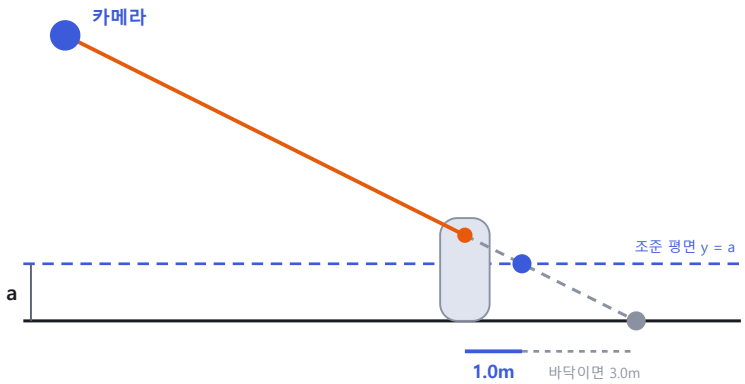
4. 남은 문제를 메우는 다섯 가지 방법

① 바닥 대신 허리 높이의 투명한 판에 찍기

쉽게

레이저가 바닥이 아니라 **총을 쏘는 높이의 판**에 찍히게 한다. 밀림이 3.0m에서 1.0m로 줄어든다. **좋은 점** 몹이 없어도 되고 튀지 않는다. **한계** 판 높이는 하나라 큰 몹 머리는 여전히 어긋난다.

자세히



바닥 대신 '쏘는 높이'와 만나는 점

$$d(a) = (h - a) \cdot D / (H - h)$$

- a = 0 이면 원래 식 그대로
- 커서 높이 h = a 에서 오차 0
- 몹이 없어도 동작하고 연속적 (튐 없음)
- 콜라이더를 안 보니 실루엣 문제와 무관

한계

a는 하나뿐. 큰 몹 머리처럼 h가 a에서 멀어지면 (h − a) 가 다시 커진다.

실무에선

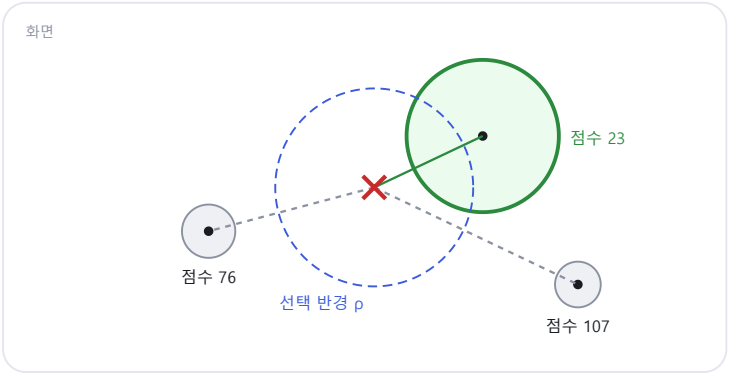
Unity 포럼의 탑다운 슈터 조준 질문에 가장 흔한 답이 "바닥 대신 총구 높이에 Plane을 두고 레이캐스트"다.

② 3D 공간 말고 "화면"에서 커서에 가까운 몹 고르기

쉽게

플레이어가 보는 **화면 위에서** 커서 근처 범위 안의 가장 가까운 몹을 고른다. **좋은 점** 카메라 각도·몹 키와 무관하고, 큰 몹은 화면에서도 커서 잘 잡힌다. **한계** 범위 경계는 남는다 → ③.

자세히



3D 레이 대신 플레이어가 보는 화면에서

$$\text{점수}_i = \| \pi(M_i) - c \| - \rho_i$$

π(M) : 몹 위치를 화면에 투영한 점
c : 커서, ρ_i : 화면에 비친 몹 반지름
선택 = 점수 최소 · 단, 점수 < ρ

- 카메라 각도·몹 키와 무관 → 원인을 우회
- 큰 몹은 ρ_i가 커서 자연히 잘 잡힌다
- 실루엣을 조금 벗어나도 잡힌다

실무에선

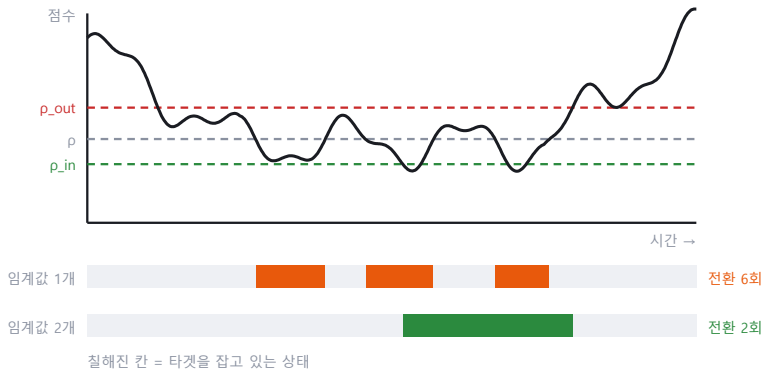
커서 방향과 적 방향의 각도를 내적으로 비교하는 콘 판정이 흔하다(gamedev.net). Hades는 맞히고 싶을 만한 적을 추린 뒤 조준 방향에 가장 가까운 적을 고르는 것으로 알려져 있다(비공식 역분석). 큰 각도로 꺾여 "조종을 뺏긴다"·"소환물을 보스보다 먼저 고른다"는 불만도 함께 나온다. **우선순위 규칙**이 품질을 가른다.

③ 자석처럼: 붙잡을 땐 깐깐하게, 놓을 땐 느슨하게

쉽게

몹을 잡는 기준과 놓는 기준을 **다르게** 둔다. 한 번 잡은 몹은 커서가 조금 흔들려도 놓지 않는다. 같은 움직임에서 조준 바뀜이 6번 → 2번으로 줄었다.

자세히



잡을 때와 놓을 때 기준을 다르게

잡기 : 점수 < ρ_in

놓기 : 점수 > ρ_out

(ρ_out > ρ_in)

- 경계에서 깜빡임이 사라진다
- = 이미 잡은 타겟에 '끈끈함'
- 레이 방식의 튐 문제도 이걸로 완화

실무에선

Path of Exile은 마우스를 누르고 있으면 그 대상이 고정돼서 커서를 움직여도 유지된다(포럼). 기준을 두 개 두는 것의 극단적인 형태가 "락온"이다.

④ 스킬마다 커서를 다르게 읽기

쉽게



총은 몹을, 장판은 바닥 그 자리를, 돌진은 방향만 원한다. **스킬 종류로 나눠 읽으면** 문제 4(조준 뺏김)가 풀린다.

실무에선

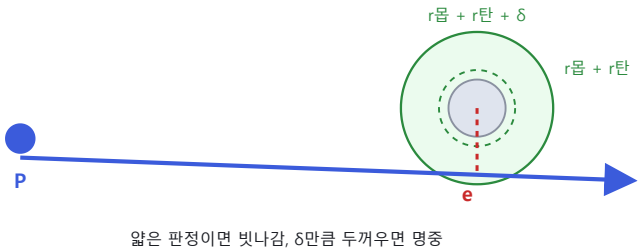
Wolcen은 "커서 밑에 뭐가 있나"를 담당하는 타겟팅 시스템을 입력 처리와 따로 둔다. 같은 커서 정보를 스킬마다 다르게 쓰기 위한 구조다.

⑤ 조준은 그대로, 과녁을 살짝 크게

쉽게

조준을 고치는 대신 **맞았는지 판정을 넉넉하게** 본다. **좋은 점** 작은 오차는 아예 못 느낀다. **한계** 너무 크면 안 맞을 걸 맞힌다.

자세히



얇은 판정이면 빗나감, δ만큼 두꺼우면 명중

조준 대신 결과를 관대하게

명중 ⇔ dist(탄 경로, M) < r몹 + r탄 + δ

- 레이 → 구(sphere) 캐스트 = 두꺼운 탄
- Reticle is King : 발사 순간 커서가 적 위면 명중으로 처리하거나 탄을 살짝 휘게

한계

δ가 크면 '안 맞을 걸 맞힌다'.
공정성(특히 PvP)과 손맛의 경계.

실무에선

TLOU2는 화살이 아깝게 빗나가는 문제를 레이캐스트 → **구 캐스트**로 바꾸고 반경을 조금 키워 풀었다. Halo·Witchfire의 bullet magnetism은 쏜 순간 탄 방향을 근처 적 쪽으로 살짝 휜다. Witchfire는 "판정을 키우는 것"과는 다르다고 강조한다. 적의 판정은 그대로 두고 보정을 **무기**에 붙인다.

5. 실무에서 더 쓰는 방법

위 다섯 가지는 실제로 쓰인다. 실무에서는 여기에 네 가지를 더 겹친다. 마우스 쿼터뷰 조준만 다룬 공식 자료는 찾지 못했다. 그래서 가장 가까운 사례를 우리 상황에 옮겨 설명한다.

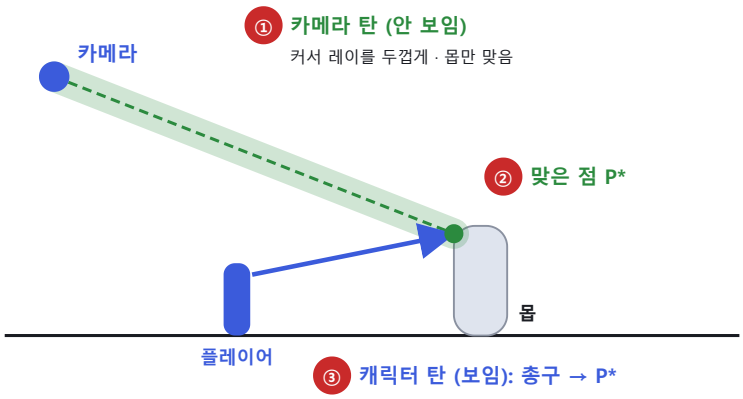
실무 1 · 판정은 카메라가, 캐릭터 탄은 연출만 (TLOU2 이중 프로브)

쉽게

TLOU2 발표는 3인칭 슈팅의 진짜 문제를 **"카메라와 캐릭터가 따로 논다"**로 정의한다. 우리 문제도 똑같다. 커서(카메라)는 몹을 보는데 탄은 캐릭터에서 나간다.

해법: **보는 쪽(카메라)이 맞았는지 정하고**, 캐릭터 탄은 그 결과를 향해 날아가는 연출로 만든다. 쿼터뷰에서는 **커서 레이가 곧 카메라 탄**이라 그대로 옮길 수 있다.

자세히



TLOU2 이중 프로브를 쿼터뷰로 옮기면

P* = 구 캐스트(커서 레이, r, 몹만)

탄 방향 = (P* − 총구) 정규화

- 판정은 '플레이어가 보는 화면' 기준
- 캐릭터 탄은 결과를 따라가는 연출
- 원작엔 벽만 검사하는 숨은 레이가 하나 더
- 안 맞으면 → ① 허리 높이 판으로

한계

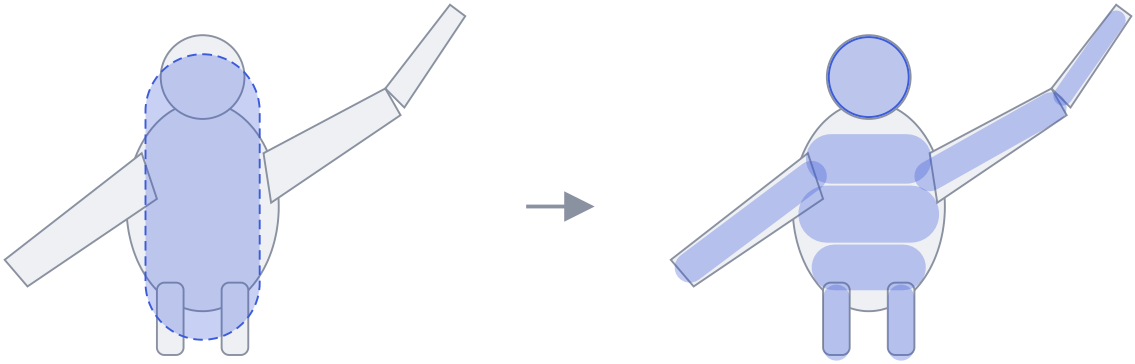
플레이어에게 유리 → 경쟁 게임엔 부적합

(발표자가 직접 밝힌 단점)

원작 구성: ① 카메라 탄 = 조준점 방향 구 캐스트, 벽 무시 · ② 숨은 레이캐스트 = 카메라 탄을 따라가며 벽만 검사 · ③ 캐릭터 탄 = 카메라 탄의 예상 충돌점에서 역산, 적에게만 충돌. 결론 슬라이드는 **"Reticle is King"**(화면에 보이는 조준이 곧 결과)이다. 우리 "몹 포함 레이"와 "판정 두께"를 하나로 합친 형태이고, 두께 덕분에 테두리에서 튀는 문제도 줄어든다.

실무 2 · 판정을 넓히지 말고 모양을 맞추기

쉽게



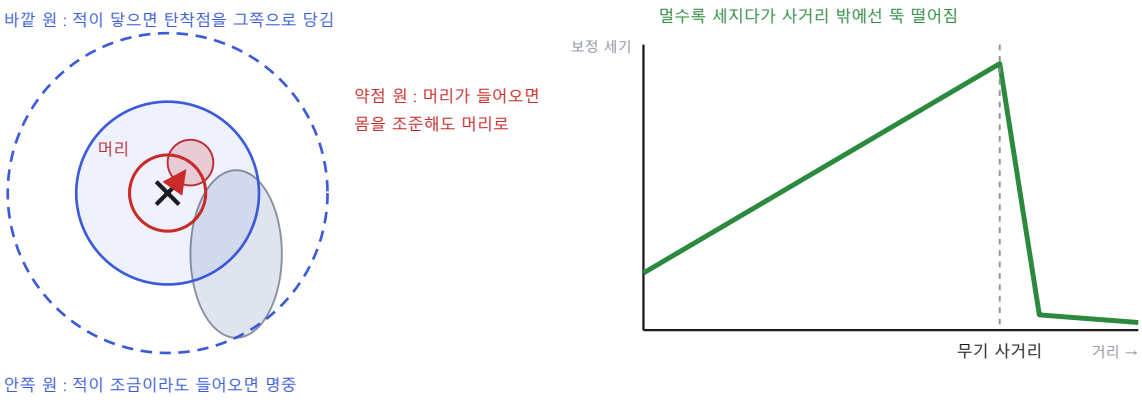
보이는 모양과 판정이 다르면 판정을 키워서 덮지 않는다. **판정 모양 자체를 몸에 맞게** 여러 개로 쪼갠다.

실무에선

TLOU2는 상체 박스 하나를 **척추 관절마다 붙인 가로 캡슐 여러 개**로 바꿔서 숙이거나 비튼 자세의 이상한 명중·빗나감을 없앴다. Witchfire는 반대로 보정 계산에는 **척추 같은 핵심 뼈만** 쓴다. 잠깐 든 손 때문에 조준이 끌려가지 않게 하려는 것이다. 맞았는지 볼 땐 정밀하게, 끌어당길 땐 단순하게.

실무 3 · 무기에 붙은 원 세 개와 약점 우선 (Witchfire)

쉽게



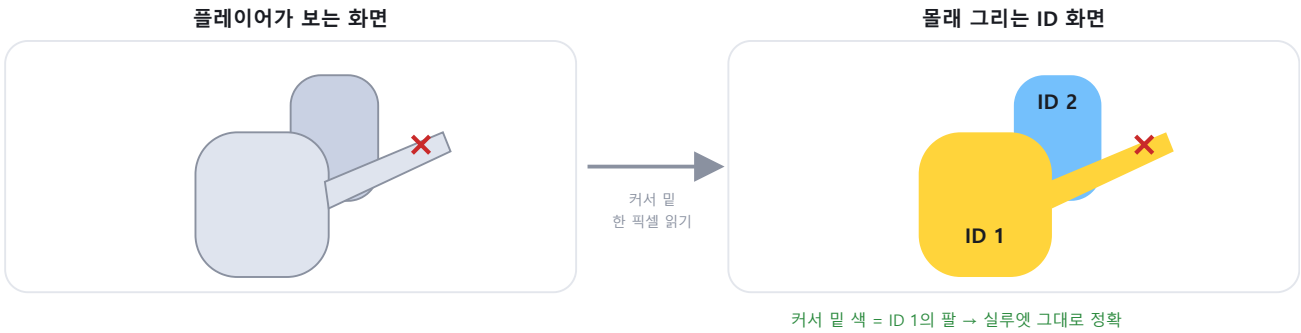
커서 둘레에 원 세 개를 둔다. 바깥 원에 닿으면 끌어당기고, 안쪽 원에 들어오면 명중이고, **약점 원에 머리가 들어오면 머리로** 보낸다. 보정 세기는 무기 사거리까지 거리에 따라 커지다가 사거리 밖에선 급히 줄어든다.

자세히

문제 3("큰 몹은 어디를 노렸나")에 대한 실무 답에 가깝다. 중심으로 끌지도, 찍은 점만 쓰지도 않고 **부위에 우선순위**를 둔다. 무기마다 원 크기와 세기가 따로 있고, 옆에서 보면 거리 감쇠 때문에 완벽하지 않은 원뿔 모양이 된다.

실무 4 · 화면 픽셀로 고르기 (ID 버퍼 피킹)

쉽게



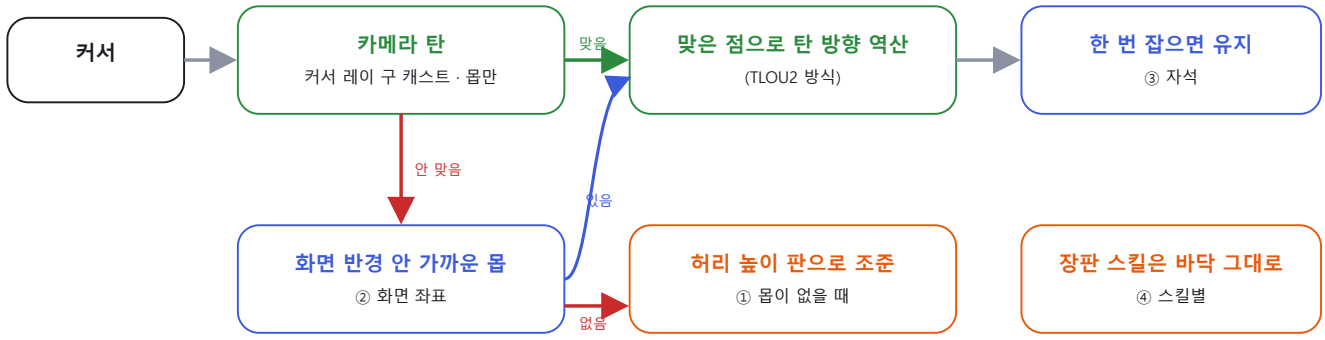
커서 밑 색 = ID 1의 팔 → 실루엣 그대로 정확

몹마다 고유한 색으로 **보이지 않는 화면을 한 장 더** 그리고, 커서 밑 한 픽셀의 색으로 무엇인지 알아낸다. 실루엣 그대로 정확하다. **한계** GPU에서 값을 읽어오는 비용과 지연 때문에 에디터(Blender)·RTS 선택에 주로 쓰고, 슈팅 판정에는 잘 쓰지 않는다. 테두리 튐 문제는 그대로 남는다.

우리와는 무관한 것

마찰(friction, 커서 감속) — 적 근처에서 조준 속도를 늦추는 보정. Halo와 Witchfire 모두 **컨트롤러 전용**이다. 마우스 쿼터뷰에는 해당하지 않는다.

6. 쿼터뷰에 옮기면: 추천 흐름

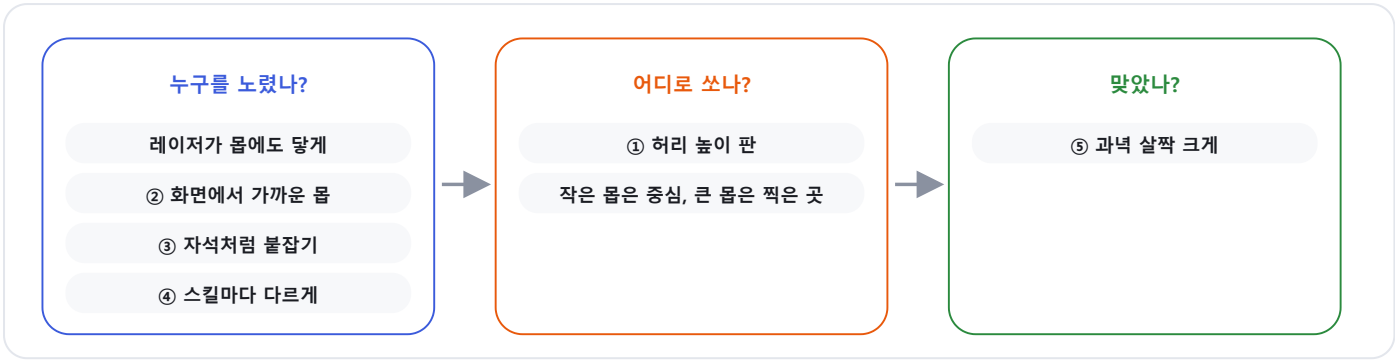


판정은 마지막에 ⑤ 살짝 넉넉하게 · 큰 몹은 부위별 캡슐로

순서대로 읽기

1. 커서 레이를 **몹만 맞는 두꺼운 구 캐스트**로 쏜다 (실무 1).
2. 맞으면 그 점을 향해 총구에서 탄 방향을 역산한다. 한 번 잡은 몹은 쉽게 놓지 않는다 (③).
3. 안 맞으면 화면 반경 안의 가까운 몹을 찾는다 (②). 그것도 없으면 허리 높이 판으로 조준한다 (①).
4. 장판 스킬은 이 흐름을 타지 않고 바닥 점을 그대로 쓴다 (④).
5. 판정은 살짝 넉넉하게 (⑤). 큰 보스는 부위별 캡슐(실무 2)과 약점 우선(실무 3)을 더한다.

7. 한 장 요약



방법	바닥 밀림·발산	테두리 튐	모양 차이	큰 몹 어디를	의도 모호
레이저가 몹에도 닿게	◐ 몹 위만	✕ 새로 생김	✕	✕ 중심 고정	✕ 흡착
① 허리 높이 판	◐ 판 근처만	● 연속	● 무관	✕	—
② 화면에서 가까운 몹	●	◐ 범위 경계	●	◐	◐ 우선순위
③ 자석 (잡기·놓기)	—	●	—	—	—
④ 스킬별 해석	—	—	—	—	●
⑤ 과녁 살짝 크게	◐ 작은 오차만	—	◐	—	—
실무 1 · 카메라 판정 + 역산	●	◐ 두께만큼	◐ 두께만큼	● 맞은 점	✕
실무 2 · 부위별 캡슐	—	—	●	◐	—
실무 3 · 원 세 개 + 약점	—	◐	—	●	—
실무 4 · 픽셀 피킹	—	✕	●	—	—

● 해결 ◐ 일부 ✕ 안 됨·악화 — 해당 없음

결론

1. "레이저가 몹에도 닿게"는 좋은 출발점이지만 문제를 **몹 테두리로 모을 뿐**이다.
2. 한 방법으로 다 막히는 칸은 없다. 실무는 **여러 개를 겹친다**.
3. 쿼터뷰에서 가장 설득력 있는 축은 **"판정은 커서(카메라)가, 탄은 연출만"**(TLOU2)이다. 커서 레이가 곧 카메라 탄이라 구조를 그대로 옮길 수 있다.

출처

1. Derek Mattson, "A Second Shot: Improving Ranged Combat in The Last of Us Part II", GDC 2024 — GDC Vault · 발표 노트 PDF (이중 프로브, 구 캐스트, 척추 캡슐, Reticle is King)
2. The Astronauts, "Witchfire aim assist" — theastronauts.com (원 세 개, 약점 우선, 거리 감쇠, 핵심 뼈만 검사, 마찰은 컨트롤러 전용)
3. Halopedia, "Aim assist" · "Magnetism" — Aim assist · Magnetism (탄 방향 보정, 감속은 컨트롤러 전용)
4. Unity Discussions — Mouse aim offset correction in a top-perspective 3D game · 3D Top Down Shooter Aiming Issue (총구 높이 평면)
5. gamedev.net — aim assist and bullets correction (콘 판정, 내적)
6. Steam 커뮤니티 — Hades targeting 토론 (비공식, 우선순위 불만)
7. Path of Exile 포럼 — Click-and-hold monster targeting (누르고 있으면 대상 고정)
8. Wolcen Update #58 — gamebanshee.com (커서 밑 감지를 입력과 분리)
9. Blender GPU select — GPU_select.h · Apple Developer Forums Picking with Metal (ID·깊이 버퍼 피킹)

쿼터뷰 조준 보정 합본 · 이론판(AimFix_Theory) + 쉬운판(AimFix_Easy) + 실무 사례 · 그림 숫자는 예시값(프로젝트 실측 아님) · 생성: docs/Combat/tools/aim_fix_full_gen.py